

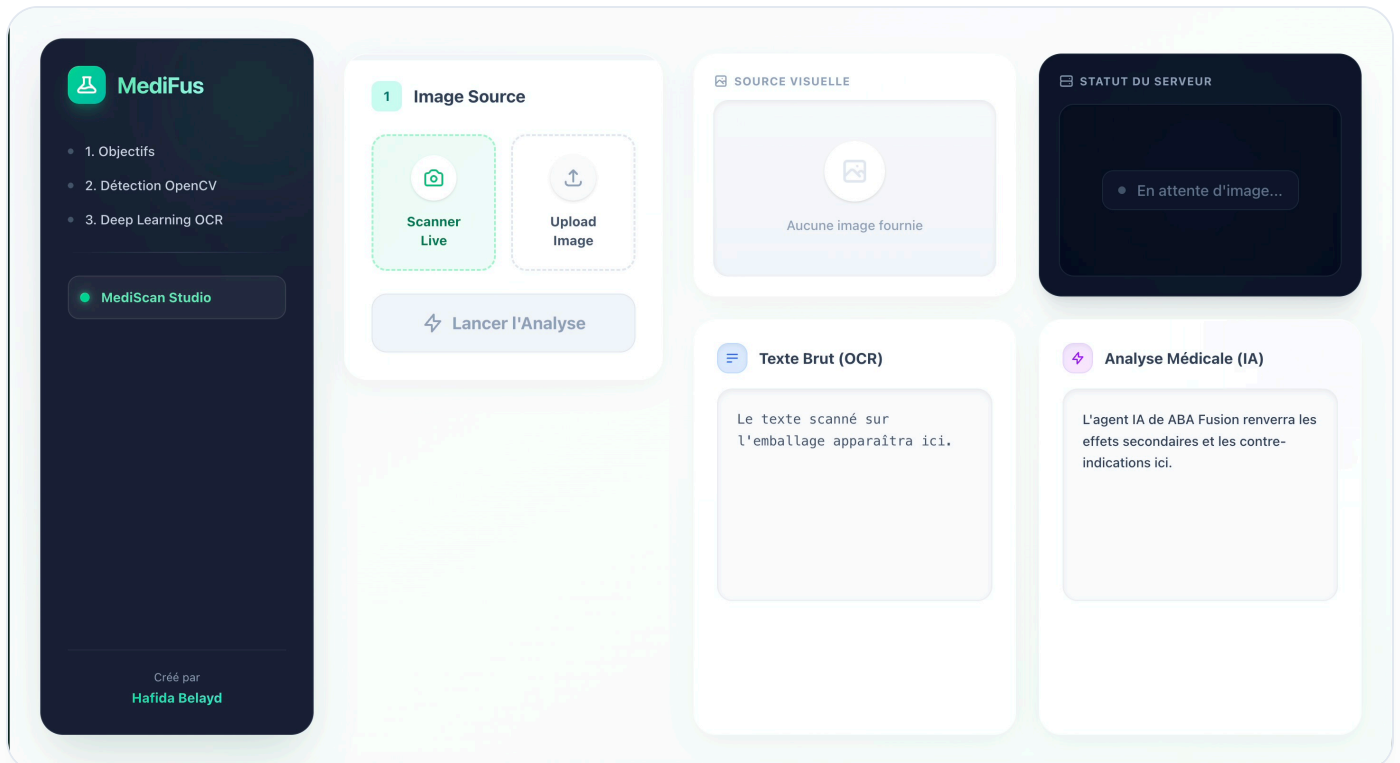


TECHNOLOGY

MediScan Lab

Documentation Officielle du système intelligent de numérisation et d'analyse des médicaments.

Créé par **Hafida Belayd**





Architecture Globale

Le projet s'appuie sur une architecture moderne séparant l'interface utilisateur de la logique serveur lourde :

- ✓ **Frontend (Dashboard)** : React.js, Vite, TailwindCSS.
- ✓ **Backend (API)** : Python, FastAPI, OpenCV, EasyOCR.



Le Flux de l'Interface (Frontend)

L'interface React orchestre le processus de capture en temps réel avec une expérience fluide. Voici comment la magie opère côté client :

1 Activation de la Caméra

L'utilisateur autorise l'accès à sa caméra (via `navigator.mediaDevices`). Le flux vidéo s'affiche en direct avec une animation de scanner "laser".

2 Boucle de Vérification Silencieuse (Auto-Capture)

Toutes les secondes, le Dashboard capture une image depuis la vidéo via un *Canvas HTML* caché et l'envoie au filtre intelligent du backend (`/detect-box`) pour voir si un médicament est devant la lentille.

3 Validation et Envoi à l'IA

Si le serveur valide l'image (`detected: true`), la boucle s'arrête ! La caméra se met en pause, et la photo est immédiatement envoyée au moteur principal (`/process-image`) pour lancer l'OCR et consulter ABA Fusion AI.

4 Affichage des Résultats

Le résultat médical est affiché à l'écran. Si le système Python indique qu'il l'a trouvé dans sa mémoire (Cache), l'interface React déclenche dynamiquement un badge visuel **CACHE HIT ⚡**. L'utilisateur peut ensuite appuyer sur la corbeille pour scanner un nouveau médicament.



Détails du Backend (main.py)

Le fichier `main.py` contient toute la logique complexe du système. Voici les 5 composants majeurs que nous y avons programmés :

1. Intégration d'EasyOCR et correction SSL

Nous utilisons le modèle Deep Learning **EasyOCR**. Pour permettre le téléchargement des poids du modèle sur macOS sans erreur de certificat, nous avons ajouté un bypass SSL sécurisé pour le contexte local.

```
import easyocr
import ssl

# Correction de l'erreur SSL sur Mac pour le téléchargement des modèles
try:
    _create_unverified_https_context = ssl._create_unverified_context
except AttributeError:
    pass
else:
    ssl._create_default_https_context = _create_unverified_https_context

# Initialisation du modèle OCR bilingue (Français, Anglais)
reader = easyocr.Reader(['fr', 'en'])
```

2. Configuration CORS (FastAPI)

Pour que l'application React (port 5173) puisse communiquer avec l'API Python (port 8000), le middleware CORS a été configuré.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Accepte les requêtes du Dashboard
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

3. Les Filtres d'Intelligence Visuelle /detect-box

Ce code empêche la caméra de prendre des visages ou des murs vides en photo. Il exige une forme rectangulaire AVEC du texte à l'intérieur.

```

@app.post("/detect-box")
async def detect_box(file: UploadFile = File(...)):
    # 1. Filtre Anti-Personne (Rejette si un visage occupe >10% de l'image)
    face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcas
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeigh
    if len(faces) > 0 and (faces[0][2] * faces[0][3] > frame_area * 0.10):
        return {"detected": False}

    # 2. Filtre Géométrique et Densité (Doit être un grand rectangle avec
    contours, _ = cv2.findContours(edged, cv2.RETR_EXTERNAL, cv2.CHAIN_AP
    for c in contours[:3]:
        approx = cv2.approxPolyDP(c, 0.05 * cv2.arcLength(c, True), True)

        if len(approx) == 4 and cv2.contourArea(approx) > 15000:
            x, y, w, h = cv2.boundingRect(approx)
            roi = edged[y:y+h, x:x+w]

            # Densité des arêtes (Texte) > 3%
            density = cv2.countNonZero(roi) / float(w * h)
            if density > 0.03:
                return {"detected": True} # Boîte de médicament validée !

```

4. Le Système de Mémoire Ultra-Rapide (Cache)

Une fois l'image validée et traitée par /process-image, ce système compare le texte scanné avec l'historique pour éviter de rappeler l'IA inutilement si le médicament a déjà été analysé.

```

extracted_text = reader.readtext(temp_file_path, detail=0, paragraph=True)
cleaned_extracted = clean_text(extracted_text) # Nettoyage du bruit OCR

# Recherche de similarité (Tolérance de 15% d'erreur OCR)
for key, cached_analysis in cache_data.items():
    similarity = difflib.SequenceMatcher(None, cleaned_extracted, key).ra
    if similarity > 0.85:
        print("[CACHE HIT] Correspondance trouvée !")
        return {
            "ai_response": cached_analysis,
            "cached": True
        }

```

5. Communication avec ABA Fusion AI

Si l'image n'est pas dans le cache, le serveur prépare les données et appelle le Webhook externe pour obtenir l'analyse médicale.

```
# Préparation du JSON
payload = {
    "event": "medicine_scan",
    "document_name": file.filename,
    "extracted_content": extracted_text,
    "timestamp": datetime.now().isoformat()
}

# Envoi de la requête au système Fusion AI
response = requests.post(FUSION_WEBHOOK_URL, data=json.dumps(payload), headers=headers)

if response.status_code in [200, 201]:
    ai_result = response.json().get("ai_analysis")

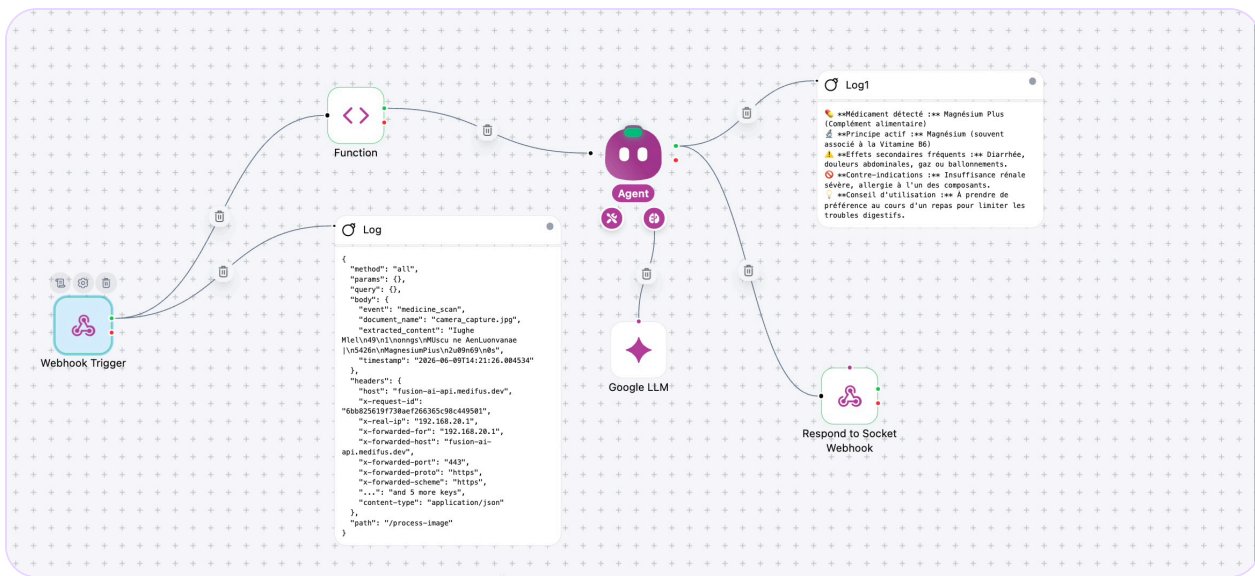
    # Sauvegarde dans le cache local pour la prochaine fois
    cache_data[cleaned_extracted] = ai_result
    save_cache(cache_data)

    return {"ai_response": ai_result, "cached": False}
```



Le Workflow de ABA Fusion AI

Lorsque le backend Python identifie une **nouvelle** boîte de médicament, il envoie le texte brut extrait par OCR vers la plateforme **ABA Fusion AI**. Le fichier JSON du workflow montre comment cette plateforme traite la requête :



1 Webhook Trigger /process-image

Le workflow écoute les requêtes HTTP entrantes (POST). Dès que notre code Python envoie le texte brut via la requête `requests.post`, ce nœud s'active et capte la donnée.

2 L'Agent IA (Pharmacien Expert)

C'est le cœur de l'analyse ! Ce nœud reçoit les données et utilise le modèle de langage **Google LLM (gemini-3-flash-preview)**. Il est programmé avec un *System Prompt* très strict :

"Tu es un pharmacien expert... Ton rôle est d'analyser le texte brut (avec fautes d'OCR) et de déduire le médicament. Formate ta réponse exactement avec : 🧴 Médicament détecté, 🧑‍⚕️ Principe actif, ⚠️ Effets secondaires, 🚫 Contre-indications, 💡 Conseil."

3

Réponse Webhook (Respond to Webhook)

Une fois que l'Agent a généré l'analyse médicale formatée et sécurisée, ce dernier nœud prend le résultat et le renvoie à notre serveur Python avec le code HTTP 200. Le Python le transmet ensuite au Dashboard React pour l'afficher à l'utilisateur.

© 2026 MediScan Lab. Tous droits réservés.

Développé avec la pile technique React & Python FastAPI.